

Ex04 — v-bind : Carte Pokémon

Informations

Notion	Liaison d'attributs — <code>v-bind (:src, :alt)</code>
Niveau	● Facile
Durée estimée	15 minutes
Travail	Individuel — sans aide IA

Objectif

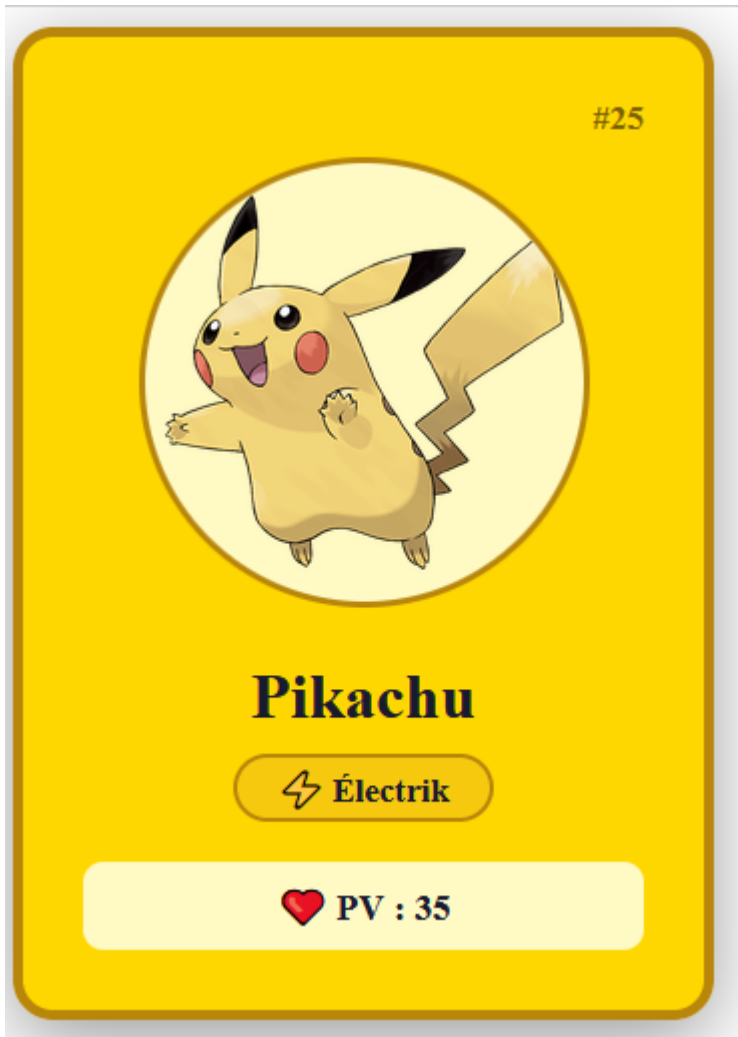
Rendre dynamique une carte Pokémon en utilisant `v-bind` pour lier l'image et ses attributs à des données réactives.

Le HTML et le CSS sont fournis — vous ne devez écrire que la **logique Vue.js**.

Structure de l'exercice

```
/ex04-carte-pokemon
|— src/
|   |— App.vue
|— index.html
```

Résultat attendu



Ce qui est fourni

Copiez ce code dans `App.vue` — le HTML et le CSS sont déjà en place. Vous n'avez qu'à compléter le `<script setup>` et modifier le `<template>`.

```
<script setup>
// À compléter
</script>

<template>
  <div class="card">
    <div class="pokedex-number">#number</div>
    <img src="" alt="" class="pokemon-image" />
    <h1>name</h1>
    <div class="type-badge">type</div>
    <div class="hp">❤ PV : hp</div>
  </div>
</template>

<style scoped>
* {
  margin: 0;
  padding: 0;
}
```

```
    box-sizing: border-box;
  }

  body {
    font-family: 'Arial', sans-serif;
    background: #1a1a2e;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
  }

  .card {
    background: #ffd700;
    border-radius: 16px;
    padding: 1.5rem;
    width: 280px;
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
    border: 4px solid #b8860b;
    text-align: center;
  }

  .pokedex-number {
    text-align: right;
    color: #7a6000;
    font-size: 0.85rem;
    font-weight: bold;
    margin-bottom: 0.5rem;
  }

  .pokemon-image {
    width: 180px;
    height: 180px;
    object-fit: contain;
    background: #fff9c4;
    border-radius: 50%;
    border: 3px solid #b8860b;
    margin-bottom: 1rem;
  }

  h1 {
    color: #1a1a2e;
    font-size: 1.6rem;
    margin-bottom: 0.5rem;
  }

  .type-badge {
    display: inline-block;
    background: #f6c90e;
    border: 2px solid #b8860b;
    border-radius: 999px;
    padding: 0.2rem 1rem;
    font-size: 0.85rem;
    font-weight: bold;
  }
```

```
color: #1a1a2e;
margin-bottom: 1rem;
}

.hp {
background: #fff9c4;
border-radius: 8px;
padding: 0.5rem 1rem;
font-size: 0.9rem;
color: #1a1a2e;
font-weight: bold;
}
</style>
```

À faire

- ✓ Définir les données réactives avec `ref()` :
 - `name` — ex : "Pikachu"
 - `number` — ex : 25
 - `type` — ex : " ⚡ Électrik"
 - `hp` — ex : 35
 - `image` — URL de l'image du Pokémon

💡 URL suggérée pour Pikachu :

<https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/official-artwork/25.png>

- ✓ Remplacer les textes en dur par `{{ }}`
- ✓ Lier `:src` et `:alt` sur la balise `<img>` avec `v-bind`
- ✓ Aucune donnée ne doit être écrite en dur dans le template
- ✗ Pas d'utilisation d'IA pour générer le code

💡 **Test** : Changez le numéro dans l'URL de l'image (ex : 25 → 1 pour Bulbizarre) et vérifiez que l'image se met à jour automatiquement.